廊桥分配

Part 1

可以枚举给国内航班分配的廊桥数量，国际航班分配的廊桥数量也随之确定。那么此时的问题就转化为廊桥数量确定的前提下，最多可以停靠的飞机数量。记录每个廊桥下一次开放的时间，根据每架飞机的到达时间排序，开放时间小于到达时间的即为空闲的廊桥，任意选择一个即可。

如果每架飞机都选择编号最小的空闲廊桥，那么增加廊桥数量时，之前已分配了空闲廊桥的飞机不受影响，因此对于不同廊桥数量下停靠的飞机数量可以一次性求解，即每个廊桥上停靠飞机数量的前缀和，时间复杂度为 $O(nm)$。

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5 + 5;
int n, m1, m2, ans;
int f1[N], f2[N];

// 核心函数：计算m架航班的廊桥分配前缀和
void solve(int m, int* f) {
    pair<int, int> p[N];
    for(int i = 0; i < m; i++) scanf("%d%d", &p[i].first, &p[i].second);
    sort(p, p + m); // 按到达时间排序

    int num = 0, t[N] = {0}; // 廊桥数量, 下一次开放时间
    for(int i = 0; i < m; i++) {

        // 找最小编号的空闲廊桥
        int a = p[i].first, b = p[i].second, now = 1;
        while(now <= num && t[now] > a) now++;

        f[now]++, t[now] = b;
        if(now > num) num = now;  // 新建廊桥
    }

    for(int i = 1; i <= n; i++) f[i] += f[i - 1];
}

int main() {
    scanf("%d%d%d", &n, &m1, &m2);
    solve(m1, f1);
    solve(m2, f2);

    // 枚举所有分配方案
    for(int i = 0; i <= n; i++) ans = max(ans, f1[i] + f2[n - i]);
    printf("%d\n", ans);
    return 0;
}
```

Part 2

在为每架飞机确定停靠位置时，需要查询满足 $t[j] < a$ 的最小 j，可以转化为数据结构计算区间最小值问题。考虑到这里的 a 是单调递增的，可以使用两个优先级队列，按结束时间排列正在使用的廊桥，按编号排列空闲的廊桥，时间复杂度为 $O((n+m)\log n)$。

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5 + 5;
int n, m1, m2, ans;
int f1[N], f2[N];
pair<int, int> p[N];

// 核心函数：计算m架航班的廊桥分配前缀和
void solve(int m, int* f) {
    pair<int, int> p[N];
    for(int i = 0; i < m; i++) scanf("%d%d", &p[i].first, &p[i].second);
    sort(p, p + m); // 按到达时间排序

    // 正在使用的廊桥，按结束时间排列
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> used;
    // 空闲的廊桥，按编号排列
    priority_queue<int, vector<int>, greater<int>> fre;

    int num = 0; // 廊桥数量
    for (int i = 0; i < m; i++) {
        int a = p[i].first, b = p[i].second, now;

        // 处理已经空闲的廊桥
        while (!used.empty() && used.top().first <= a)
            fre.push(used.top().second), used.pop();

        if (!fre.empty())
            now = fre.top(), fre.pop();
        else
            now = ++num;  //新建廊桥

        f[now]++, used.push({b, now});
    }

    for(int i = 1; i <= n; i++) f[i] += f[i - 1];
}

int main() {
    scanf("%d%d%d", &n, &m1, &m2);
    solve(m1, f1);
    solve(m2, f2);

    // 枚举所有分配方案
    for (int i = 0; i <= n; i++) ans = max(ans, f1[i] + f2[n-i]);
    printf("%d\n", ans);
    return 0;
}
```

Part 3

可以依次考虑每个廊桥上可以停靠哪些飞机，根据先到先得的规则，当前廊桥空闲之后首个到达的飞机会停靠在此，也就是满足 $t[j] < a$ 的最小 a，可以使用平衡树实现，时间复杂度为 $O((n+m)\log m)$。

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5 + 5;
int n, m1, m2, ans;
int f1[N], f2[N];

// 核心函数：计算m架航班的廊桥分配前缀和
void solve(int m, int* f) {
    int a, b;
    map<int, int> S; // 使用平衡树存储
    for(int i = 0; i < m; i++) scanf("%d%d", &a, &b), S[a] = b;

    // 计算每个廊桥停靠的飞机
    f[0] = 0;
    for(int i = 1; i <= n; i++) {
        f[i] = f[i - 1];
        auto it = S.begin(); // 首架飞机直接停靠
        while(it != S.end()) {
            f[i]++;
            b = it->second;
            S.erase(it);
            it = S.upper_bound(b); //查询下一架可停靠飞机
        }
    }
}

int main() {
    scanf("%d%d%d", &n, &m1, &m2);
    solve(m1, f1);
    solve(m2, f2);

    // 枚举所有分配方案
    for(int i = 0; i <= n; i++) ans = max(ans, f1[i] + f2[n - i]);
    printf("%d\n", ans);
    return 0;
}
```